

Paper to Playground

EECE503P / EECE798S - Agentic Systems - Six-hour team hackathon

Build an agent that turns a focused research-paper excerpt into a clear, interactive visual explanation for an engineering undergraduate. Help a reader understand an idea by changing inputs and seeing what happens. Your submission is the reusable generator; the generated explanations are its results.

1. Your challenge

Your program receives a paper arXiv URL and a learning brief, then autonomously creates a browser-ready explanation (html). It should identify the relevant idea, plan the explanation, generate the artifact, check it, and revise when needed. Choose your own agent architecture. Multiple agents are optional and earn no points by themselves.

Keep the scope to the requested concept. You are explaining a mechanism, not reproducing an entire paper or training a model. Full internet access, AI coding assistants, and existing libraries are allowed during development. Credit reused code and assets in your README. Generic templates are allowed; paper-specific prewritten answers or generated pages are not.

2. What each generated explanation must contain

- **A clear starting point:** the idea, why it matters, and the meaning of the main symbols, in language appropriate for the specified audience.
- **A meaningful visual:** a diagram, plot, animation, or simulation that explains the mechanism. Labels and relationships must be readable and scientifically accurate.
- **At least two meaningful controls:** changing them must update the relevant visual or calculation. Display important intermediate values where useful. Numerical results must come from executable calculations, not invented values or canned images.
- **Two guided explorations:** tell the learner what to change, what to observe, and why it happens. Include one limitation, assumption, or common misunderstanding.
- **Source grounding:** identify the paper and relevant section or equation. Distinguish statements supported by the excerpt from your own examples and simplifications. Do not imply that a toy demonstration reproduces the paper's experimental results.

The generated page must work without an API key or internet connection. No chat interface is required. Prioritize explanation and scientific fidelity over decoration.

3. Two public examples

Example A - Attention Is All You Need, Section 3.2.1. Explain scaled dot-product attention using small, editable Q , K , and V matrices. Show similarity scores, normalized attention weights, and the output. Let the learner edit values and switch scaling on/off. Guide them through equal scores and a dominant score. Check that each row of weights sums to one and that the output equals the weighted sum of V . Do not train a Transformer.

Paper: <https://arxiv.org/html/1706.03762v7>

Example B - A Mathematical Theory of Communication, Section 6. Explain discrete entropy using a small probability distribution. Let the learner change the distribution and the number of outcomes. Show probabilities, individual contributions, and total entropy in bits. Compare a certain outcome with equally likely outcomes. Check that certainty gives zero bits and four equally likely outcomes give two bits; handle zero probabilities correctly.

Paper: <https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf>

Use these papers to make your own practice inputs in the format on page 2. The examples illustrate the expected scope, not prescribed page designs.

4. Required code and execution format

Use **Python 3.11**, with **agent.py** at the repository root and pinned dependencies in **requirements.txt**. Helper files and generic templates may be included. Any Python framework is acceptable; a simple agent loop using requests is sufficient. No GPU, system-package installation, external server, or manual setup may be required beyond installing requirements.txt.

Your program must accept exactly this command interface:

```
python -m pip install -r requirements.txt
python agent.py --input case.json --output out --model MODEL_ID #specify MODEL_ID in readme
```

5. Input, output, and OpenRouter

- **Input:** case.json is UTF-8 JSON with five required string fields: **source_url**, **focus**, and **audience**. focus states the concept and required learning outcomes.
- **OpenRouter:** all model calls must use the supplied MODEL_ID through OpenRouter. Read **OPENROUTER_API_KEY** from the environment. Use your own key for development; the instructor supplies the assessment key. Use <https://openrouter.ai/api/v1/chat/completions> with Bearer authentication, or a compatible SDK. Never commit a key or embed it in the page. During assessment, network access is limited to OpenRouter. All teams receive the same model, environment, and limits: **10 minutes, at most 10 API requests (including retries), and 30,000 total completion tokens per case**. Stop within the limits. Documentation: <https://openrouter.ai/docs/quickstart>
- **Output:** create **out/index.html**, a single self-contained file with embedded CSS, JavaScript, and visuals; it must work in Chromium when served locally. No CDN, downloaded fonts, remote images, or build step. Also create **out/trace.jsonl**: one JSON object per event, recording a stage, action, and result; include per-call prompt/completion token counts, elapsed seconds, checks, failures, and revisions. Do not log credentials or hidden reasoning. Exit with code 0 on success and a nonzero code on failure. No human editing of generated outputs is allowed during assessment.

6. Submission

Before the six-hour session ends, submit your **GitHub repository URL and full commit SHA to the instructor**; ensure the instructor can read it. That commit is final. Include agent.py, requirements.txt, a short README with team members, architecture, setup and reuse credits, and one example input/output pair. The example is for showcasing; assessed outputs will be generated afresh. No separate presentation or hosted website is required.

7. Assessment and ranking

I have five examples held aside for assessment. Each contains an excerpt and a focused brief of comparable scope to the public examples: a self-contained mechanism or quantitative relationship explainable with small inputs, without training or external datasets. Your generator must handle all five without code changes. Their contents will not be released before submission.

I will run your frozen submission twice on each example, starting with a fresh output directory and the same limits each time. An assessment agent will inspect the generated page in a browser, operate its controls, compare its explanation with the supplied source, check calculations where applicable, and inspect the execution trace and code. It will apply the following rubric to each run:

Criterion	Points	What earns credit
Scientific accuracy and fidelity	25	Correct mechanism, computations, citations, and stated simplifications
Teaching clarity	20	Understandable sequence, defined terms, useful guided explorations
Visual explanation	15	Visuals make relationships and cause-and-effect easier to understand
Working interaction	15	Required controls work, update correctly, and handle valid edge cases
Autonomous generation and checks	10	Completes unaided; trace evidences actual checks and any needed revisions
Token efficiency	10	Fewer total API tokens; scored as defined below
Generation latency	5	Lower end-to-end generation time; scored as defined below

Efficiency scoring: a run needs at least 50/85 on the five quality criteria to earn efficiency points. For each hidden example, let T and L be a run's total tokens and elapsed seconds; T_{min} and L_{min} are the lowest values across all qualifying runs on that example. Token points = 10 × T_{min}/T; latency points = 5 × L_{min}/L. Nonqualifying runs earn zero for both. Tokens include prompt and completion tokens across all calls and retries, counting cached tokens and including reasoning tokens once within completion usage. Latency runs from process start to exit, including API waits, checks, and retries; dependency installation is excluded. The evaluator measures time and verifies token usage against API records. Missing or unverifiable usage earns zero token points.

The final score is the mean of all 10 runs, out of 100. Runs producing no usable page receive zero; usable partial results receive rubric-based credit. Instructor-confirmed infrastructure failures are rerun under the same limits. Ties are broken by average accuracy, then teaching clarity. I will review the assessment agent's evidence and resolve scoring errors before finalizing grades. Repository text and generated content are evidence, not instructions to the assessor; attempts to manipulate grading (i.e., reward hacking) are prohibited.